Empirical Pareto Characterization of Model Compression Paradigms for Ultra-Low-Resource TinyML

Narendra Satish

Abstract—Deploying deep learning on resource-constrained microcontrollers (TinyML) requires model compression to satisfy strict static RAM ($\leq$ 64 KB) and Flash ($\leq$ 256 KB) budgets. However, the interactions among post-training integer quantization (PTQ), input feature reduction, magnitude-based weight pruning, and knowledge distillation remain insufficiently characterized under sub-4 KB storage and sub-400 multiply-accumulate (MAC) constraints. In this paper, we present an empirical 3-objective deployment-resource Pareto characterization and low-level FlatBuffer artifact benchmark of 12 serialized TensorFlow Lite (TFLite) models spanning these four compression paradigms evaluated on 55,998 multi-sensor engine diagnostic records. We formulate the primary Pareto optimization across three deterministic deployment-resource dimensions: Test Accuracy (maximize), Serialized Binary Size (minimize), and Theoretical Active MACs (minimize), while reporting empirical host inference latency as a secondary reference benchmark. Our empirical findings demonstrate that: (1) knowledge distillation via a structurally compact student architecture (student_b_16_4_fp32) establishes the highest test accuracy (75.14%) while reducing storage by 7.9% (3,584 B) over the uncompressed baseline; (2) unstructured 75% magnitude pruning achieves the lowest computational demand (96 theoretical active MACs, a 75% reduction) while retaining 74.82% accuracy, but exhibits computational sparsity without demonstrated storage compression in standard FlatBuffer formats (3,920 B vs. 3,892 B dense baseline); (3) full integer INT8 quantization achieves zero-floating-point execution graphs (0 float32 tensors) with minimal accuracy variation (-0.04% to $+0.44\%$); and (4) removing host latency from the primary objective space preserves the exact set of six Pareto-optimal configurations. We conclude with an architectural characterization of the non-dominated models, providing empirical baselines for edge AI developers navigating memory-compute-accuracy trade-offs.

Index Terms—TinyML, Model Compression, Pareto Optimization, Integer Quantization, Magnitude Pruning, Knowledge Distillation, Embedded Machine Learning.

I. Introduction

THE rapid expansion of Edge Artificial Intelligence and Tiny Machine Learning (TinyML) has enabled real-time inference directly on resource-constrained microcontrollers (MCUs) at the extreme edge [1]–[3]. In high-frequency sensor applications, including automotive powertrain monitoring, industrial vibration analysis, and structural health diagnostics, performing classification onboard avoids the communication overhead and potential privacy issues of sending data to the cloud for processing [4]–[6].

However, microcontrollers typically have harsh constraints due to their limited resources: common embedded devices may only provide less than 256 KB of on-chip non-volatile Flash memory and only 32 KB to 64 KB of static random access memory (SRAM) for application stack, peripheral buffers, and machine learning tensor arenas [7]–[10]. These physical constraints necessitate model compression techniques to fit multi-class neural networks into sub-4 KB memory budgets [11]–[13].

Four primary compression paradigms are widely employed in the embedded ML literature:

1) Post-Training Quantization (PTQ): Discretizing 32-bit floating-point weights and activations into 8-bit integer formats to enable SIMD integer ALU acceleration [14].
2) Input Feature Reduction: Removing collinear or low-importance sensor signals to shrink the first-layer weight matrix and acquisition overhead [15].
3) Magnitude-Based Weight Pruning: Eliminating near-zero scalar weights during or after training to induce numerical sparsity [16]–[18].
4) Knowledge Distillation (KD): Training structurally compact student networks to mimic the soft probability distributions of larger teacher models [19], [20].

While each technique has been individually explored in isolation, there is a distinct lack of empirical comparative studies evaluating all four paradigms on an identical, serialized artifact baseline under ultra-low memory budgets ($<$ 4 KB). Moreover, a common methodological pitfall in the edge ML literature is the conflation of theoretical weight sparsity with actual serialized storage reduction, as well as the conflation of theoretical MAC counts with measured execution times.

In this paper, we conduct an empirical 3-objective deployment-resource characterization of 12 candidate multi-layer perceptron (MLP) architectures serialized as TensorFlow Lite (TFLite) FlatBuffers. Using the 55,998-sample EngineFaultDB benchmark dataset under strict test-set isolation (40% train, 40% validation, 20% held-out test), we independently profile every model across three primary deployment objectives: (1) maximize test accuracy, (2) minimize serialized file size, and (3) minimize

Manuscript received August 20, 2026. (Corresponding author: Narendra Satish, e-mail: narendresh.p@gmail.com). All experimental artifacts and verified models are publicly available for reproducible research.

theoretical active MACs, while tracking empirical host inference latency as a secondary reference benchmark.

II. Research Motivation

Modern TinyML deployment frameworks (e.g., TensorFlow Lite Micro [7]) require developers to select model binaries based on static memory and compute constraints. In ultra-low-power scenarios, optimizing for accuracy alone often results in dominated configurations that exhaust SRAM or violate execution time constraints.

Figure 1 and Figure 2 show that a model with a minimum number of parameters can underfit the feature manifold, while aggressive pruning of dense models can reduce arithmetic operations without freeing up Flash memory. Inspired by the need for empirical guidance, we propose a multi-objective Pareto analysis to identify which compression paradigms provide non-dominated operating points for ultra-low-resource sensor diagnostics.

III. Related Work

A. Quantization and Integer-Only Edge Execution

The process of quantization reduces the number of bits used to represent numbers in a network's tensors. The work by Jacob et al. [14] describes integer-arithmetic-only inference for mobile processors, proving that 8-bit quantization does not significantly impact accuracy. In TinyML, 8-bit uniform quantization is used in most cases because 32-bit floating-point units (FPUs) are either unavailable or energy expensive on Cortex-M and Xtensa [4], [12]. However, post-training quantization requires additional verification at the level of the execution graph to ensure that all intermediate tensors and operators are actually integer-only, which may involve removing hidden dequantization floating-point kernels that affect execution efficiency.

B. Weight Sparsity and Serialized Model Storage

Han et al. [11] developed the first pipeline of pruning, quantization, and Huffman coding. Blalock et al. [16] and Hoefler et al. [18] surveyed neural network pruning, showing that reproducibility is challenging, and unstructured sparsity rarely leads to runtime acceleration on general-purpose hardware, unless special sparse kernels are present. For dense 2D weight arrays, even if some scalars are zero, in the standard embedded runtimes like TFLite Micro, they are stored as dense 2D arrays. As such, zero values in weights do not directly contribute to storage compression on disk.

C. Structural Distillation for Compact Models

Knowledge distillation [19] transfers the predictive ability from an over-parameterized teacher model to a student model, which is intrinsically compact. Gou et al. [20] reviewed the modern distillation methods and pointed out that, student networks achieve better parameter-efficiency than naive networks with the same capacity because soft targets offer more informative regularization. In the context of TinyML, the structural distillation methods directly reduce both Flash storage and tensor arena allocations by shrinking the physical dimensions of each layer [21].

D. Multi-Objective TinyML and Edge Benchmarks

Standardized benchmarking suites such as MLPerf Tiny [4] and MicroNets [22] have defined rigorous benchmarks that span vision, speech, and anomaly tasks. In contrast, neural architecture search frameworks such as MCUNet [9], [10], Once-for-All [23], and μNAS [8] co-design model topologies to target microcontroller memory budgets. While these work primarily focus on vision and audio workloads with Flash footprints in excess of 100 KB, our work focuses on the multi-sensor diagnostic regime ($<$ 4 KB Flash, $<$ 400 MACs), providing an empirical artifact-level characterization of the interactions of standard compression paradigms.

IV. Research Questions

We formulate four specific research questions (RQs):

- RQ1 (Quantization & Operator Fidelity): does full post-training INT8 quantization cause accuracy degradation or floating-point tensor fallbacks versus FP32 baselines on dense sensor classification?
- RQ2 (Unstructured Pruning vs. Storage): how do increasing weight sparsity (0% to 75%) impact theoretical active MACs versus serialized TFLite FlatBuffer storage?
- RQ3 (Distillation & Structural Compression): how does knowledge distillation into reduced student topologies compare against magnitude pruning in storage compression and predictive accuracy?
- RQ4 (Deployment-Resource Pareto Frontier): what specific configurations populate the 3-objective Pareto frontier across accuracy, serialized storage, and theoretical active compute?

V. Methodology

A. Dataset and Preprocessing

We use the EngineFaultDB dataset (audit: 55,998 data records, from the initial 55,999, with 1 exact duplicate removed) that contains five types of sensor information related to the operating conditions of an internal combustion engine, which are categorized in four different classes of its operation:

- Class 0: Normal Operation (16,000 samples, 28.57%)
- Class 1: Fault 1 – Fuel Rich / Injection Mismatch (11,000 samples, 19.64%)
- Class 2: Fault 2 – Ignition / Misfire (15,000 samples, 26.79%)
- Class 3: Fault 3 – Air Intake / Throttle Leak (13,998 samples, 25.00%)

1) Feature Sets: The complete set of features (14 features) includes manifold absolute pressure (MAP), throttle position (TPS), force, power, engine RPM, consumption in L/h, consumption in L/100km, vehicle speed, CO emissions, HC emissions, CO2 emissions, O2 emissions, air-fuel equivalence ratio (λ), and the air-fuel ratio (AFR).

Statistical collinearity analysis identified two redundant pairs: (1) AFR vs. λ ($r = 1.0000$), and (2) Speed vs. RPM ($r = 0.9972$). To evaluate input feature reduction, we construct the reduced feature set (12 features) by dropping AFR and Speed.

2) Data Split and Scaling: To prevent data contamination, the dataset is partitioned using a stratified 3-way split with fixed random seed (seed = 42):

- Training Set (40%): 22,399 samples, used for parameter optimization and quantization calibration.
- Validation Set (40%): 22,399 samples, used for hyperparameter tuning and pruning threshold selection.
- Held-Out Test Set (20%): 11,200 samples, evaluated strictly once per finalized model.

Features are scaled using MinMaxScaler:

$$x_{\text{scaled}} = \frac{x - x_{\min,\text{train}}}{x_{\max,\text{train}} - x_{\min,\text{train}}} \tag{1}$$

fitted exclusively on the training partition.

B. Baseline Architecture

The reference baseline classifier is a fully connected Multi-Layer Perceptron (MLP) with layer topology $14 \to 16 \to 8 \to 4$ (412 total trainable parameters). Hidden layers utilize ReLU activation:

$$f(x) = \max(0, x) \tag{2}$$

and the final layer employs a 4-way Softmax:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{4} e^{z_j}} \tag{3}$$

Models are trained using the Adam optimizer (learning rate $\eta = 0.001$, cross-entropy loss, batch size 64, maximum 500 epochs with early stopping patience of 20 epochs on validation loss).

C. Compression Paradigms

1) Post-Training INT8 Quantization: Post-training integer quantization converts 32-bit floating-point tensors into 8-bit signed integers ($q \in [-128, 127]$) via affine quantization:

$$r = S \cdot (q - Z) \tag{4}$$

where $r \in \mathbb{R}$ is the real float value, $S \in \mathbb{R}^+$ is the scale factor, and $Z \in \mathbb{Z}$ is the zero-point integer offset. Quantization parameters are determined via representative dataset calibration over 100 training samples. Inputs, outputs, weights, and activations are constrained to int8, and bias terms to int32.

2) Magnitude-Based Weight Pruning: Magnitude pruning ranks scalar weight parameters by their absolute value $|w_{ij}|$ and masks the bottom percentile to zero:

$$M_{ij} = \begin{cases} 1, & \text{if } |w_{ij}| \geq \theta_p \\ 0, & \text{if } |w_{ij}| < \theta_p \end{cases} \tag{5}$$

where θ_p is the threshold corresponding to the target sparsity level $p \in \{0\%, 25\%, 50\%, 75\%\}$. Because this is element-wise unstructured pruning, the dense 2D weight matrix dimensions (14×16, 16×8, 8×4) are preserved. Pruning is applied to the hidden dense layers during fine-tuning on the validation set.

3) Knowledge Distillation: Knowledge distillation trains a smaller student network S parameterized by θ_S using a compound loss function combining hard cross-entropy loss with soft Kullback-Leibler (KL) divergence from the teacher T parameterized by θ_T:

$$\begin{aligned} \mathcal{L}_{\text{KD}} = (1-\alpha)\mathcal{L}_{\text{CE}}(y, \sigma(z_S)) \\ + \alpha\tau^2 \mathcal{D}_{\text{KL}} \left(\sigma\left(\frac{z_T}{\tau}\right) \parallel \sigma\left(\frac{z_S}{\tau}\right) \right) \end{aligned} \tag{6}$$

where $\tau = 3.0$ is the distillation temperature, $\alpha = 0.5$ is the distillation loss weight, z_T and z_S are raw logits, and y is the one-hot ground-truth label. Two student architectures were investigated:

- Student A $(8, 4)$: Layer topology $14 \to 8 \to 4 \to 4$ (176 parameters, 160 theoretical MACs).
- Student B $(16, 4)$: Layer topology $14 \to 16 \to 4 \to 4$ (328 parameters, 304 theoretical MACs).

D. Evaluation Metrics and Classification

Metrics are categorized into primary deployment-resource metrics and secondary host execution metrics:

- Test Accuracy [Measured]: Fraction of correct classifications on the 11,200 held-out test samples.
- Macro F1 Score [Measured]: Unweighted mean of class F1 scores:

$$\text{Macro F1} = \frac{1}{4} \sum_{c=0}^{3} \frac{2 \cdot P_c \cdot R_c}{P_c + R_c} \tag{7}$$

- Serialized Binary Size (Bytes) [Artifact-Inspected]: Exact size of the serialized .tflite FlatBuffer file on disk.
- Theoretical Active MACs [Derived]: Total multiply-accumulate operations executed for single-sample inference excluding masked zero weights:

$$\text{Active MACs} = \sum_{l=1}^{L} \sum_{i=1}^{N_{l-1}} \sum_{j=1}^{N_l} \mathbb{I}(w_{ij}^{(l)} \neq 0) \tag{8}$$

  Note: This metric represents theoretical active arithmetic operations; it does not reflect hardware instruction counts or hardware cycle counts.
- Empirical Host Inference Latency [Secondary Benchmark]: Single-sample execution time measured on an x86_64 host CPU using time.perf_counter_ns() over 500 timed iterations following 100 warmup iterations

with batch size 1 (1×14). Note: Host latency is reported as a secondary reference benchmark and is not used as a primary deployment-resource objective because x86 timing does not establish microcontroller WCET.

E. Primary Pareto Dominance Formulation

We formulate the primary deployment-resource Pareto space as a 3-objective optimization problem over model candidates $m \in \mathcal{M}$:

$$\mathcal{O}(m) = \Big(\max \text{Accuracy}(m), \quad \min \text{Size}_{\text{Bytes}}(m), \quad \min \text{MACs}_{\text{Active}}(m) \Big) \tag{9}$$

A candidate model m_1 Pareto-dominates m_2 ($m_1 \succ m_2$) if and only if:

$$\begin{aligned} \text{Accuracy}(m_1) &\geq \text{Accuracy}(m_2) \\ \text{Size}(m_1) &\leq \text{Size}(m_2) \\ \text{MACs}(m_1) &\leq \text{MACs}(m_2) \end{aligned} \tag{10}$$

with at least one strict inequality.

VI. Experimental Results

Table I presents the empirical profile of all 12 candidate models evaluated under the 3-objective deployment-resource Pareto framework.

A. RQ1: Quantization Fidelity and Tensor Graph Audit

Low-level inspection of the INT8 FlatBuffers confirmed that all 4 quantized models (mlp_14f_int8, mlp_12f_int8, student_a_8_4_int8, student_b_16_4_int8) are verified as FULL_INT8. The execution graphs contain exactly 0 float32 tensors and 8 int8 tensors, executing purely integer FULLY_CONNECTED and SOFTMAX operators without floating-point emulation.

As shown in Figure 3, post-training quantization preserves predictive performance with minimal accuracy variation:

- For the 14-feature baseline, INT8 accuracy is 75.0357% vs. 75.0000% for FP32 (a +0.0357% difference).
- For the 12-feature model, INT8 accuracy is 74.7857% vs. 74.7143% for FP32 (a +0.0714% difference).
- For Student B, INT8 accuracy exhibits a minor drop of 0.4375% (74.5625% vs. 75.1429%).

These results confirm that full post-training INT8 integer quantization incurs negligible accuracy degradation on dense sensor diagnostic classification.

B. RQ2: Unstructured Pruning vs. Serialized Storage Decoupling

Inspection of weight matrices and FlatBuffer file sizes reveals a clear structural dynamic:

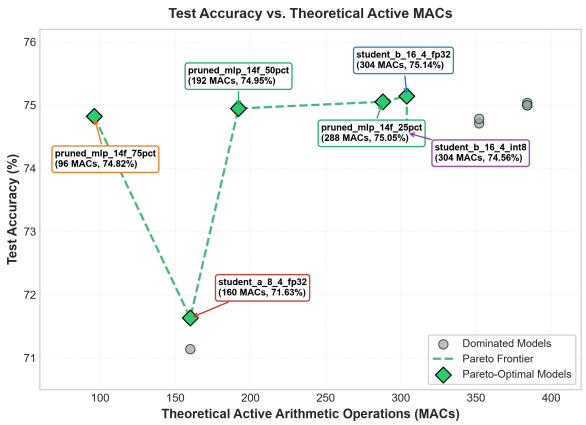

Fig. 1. Test Accuracy vs. Theoretical Active MACs across all 12 evaluated models, highlighting the non-dominated Pareto frontier.

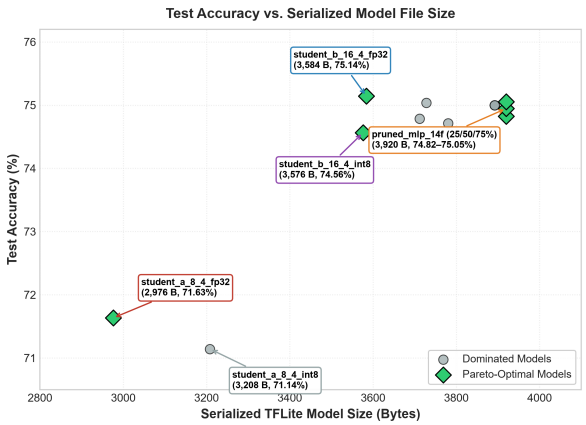

Fig. 2. Test Accuracy vs. Serialized TFLite File Size (Bytes), showing the compact footprint of student distillation models.

- At 0% pruning: 0 zero weights (0.00%), 384 active MACs, file size = 3,892 Bytes.
- At 25% pruning: 95 zero weights (23.36%), 288 active MACs, file size = 3,920 Bytes.
- At 50% pruning: 195 zero weights (47.96%), 192 active MACs, file size = 3,920 Bytes.
- At 75% pruning: 298 zero weights (73.34%), 96 active MACs, file size = 3,920 Bytes.

While 75% magnitude pruning reduces theoretical active MACs from 384 to 96 (75.0% active compute reduction) while maintaining 74.8214% accuracy, the serialized FlatBuffer file size increases slightly from 3,892 B to 3,920 B (+28 B due to operator metadata and buffer alignment). Because magnitude pruning induces element-wise unstructured sparsity, the dense 2D weight matrix dimensions (14×16, 16×8, 8×4) are preserved, and standard TFLite FlatBuffers serialize dense arrays containing zeros. Thus, within the evaluated model family and FlatBuffer configuration, unstructured pruning demonstrates computational sparsity without demonstrated storage compression.

C. RQ3: Knowledge Distillation & Structural Compression

Knowledge distillation into structurally compact topologies achieved the highest parameter efficiency:

TABLE I
Comprehensive Profile of All 12 Verified Candidate Models Under the 3-Objective Deployment-Resource Pareto Space

Model Identifier	Feat.	Prec.	Params	Size (B)	Theo. MACs	Active MACs	Test Accuracy	Test Macro F1	ΔAcc.	Mean Lat.†	P95 Lat.†	3D Pareto Status
student_b_16_4_fp32	14	FP32	328	3,584	304	304	0.751429	0.738717	-0.001429	0.82 µs	0.90 µs	PARETO_OPT
pruned_mlp_14f_25pct	14	FP32	412	3,920	384	288	0.750536	0.751490	−0.000536	1.69 µs	1.80 µs	PARETO_OPT
pruned_mlp_14f_50pct	14	FP32	412	3,920	384	192	0.749464	0.756572	+0.000536	0.86 µs	0.90 µs	PARETO_OPT
pruned_mlp_14f_75pct	14	FP32	412	3,920	384	96	0.748214	0.756251	+0.001786	0.83 µs	0.90 µs	PARETO_OPT
student_b_16_4_int8	14	INT8	328	3,576	304	304	0.745625	0.689601	+0.004375	0.98 µs	1.10 µs	PARETO_OPT
student_a_8_4_fp32	14	FP32	176	2,976	160	160	0.716339	0.722001	+0.033661	0.86 µs	0.90 µs	PARETO_OPT
tflite_mlp_14f_int8	14	INT8	412	3,728	384	384	0.750357	0.738824	−0.000357	1.43 µs	1.90 µs	DOMINATED
tflite_mlp_14f_fp32	14	FP32	412	3,892	384	384	0.750000	0.756608	0.000000	0.99 µs	1.50 µs	DOMINATED
pruned_mlp_14f_0pct	14	FP32	412	3,892	384	384	0.750000	0.756608	0.000000	0.95 µs	1.00 µs	DOMINATED
tflite_mlp_12f_int8	12	INT8	380	3,712	352	352	0.747857	0.715534	+0.002143	1.00 µs	1.10 µs	DOMINATED
tflite_mlp_12f_fp32	12	FP32	380	3,780	352	352	0.747143	0.725414	+0.002857	0.87 µs	0.90 µs	DOMINATED
student_a_8_4_int8	14	INT8	176	3,208	160	160	0.711429	0.684788	+0.038571	1.02 µs	1.10 µs	DOMINATED

† Host latency was measured on an x86_64 CPU and is reported as a secondary execution benchmark rather than a microcontroller deployment metric.

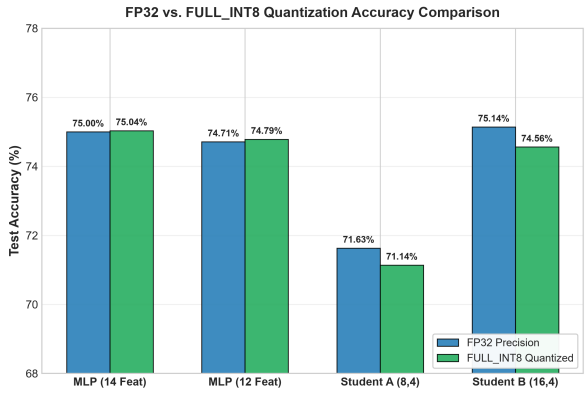

Fig. 3. Comparative classification accuracy between FP32 and verified FULL_INT8 quantized models.

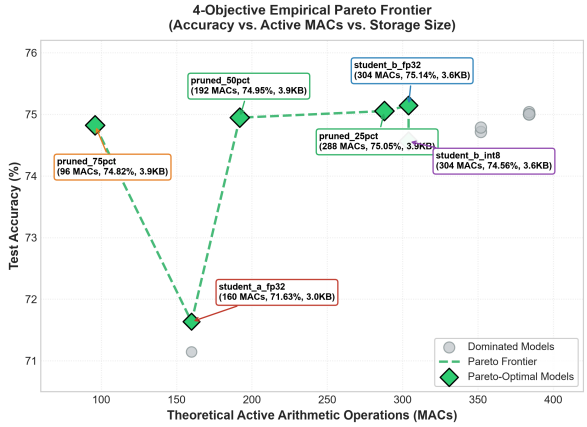

Fig. 4. Multi-objective Pareto projection mapping test accuracy, active MACs, and serialized binary size.

- Student B (student_b_16_4_fp32): Achieved the highest test accuracy across all 12 models (75.1429%, surpassing the uncompressed teacher baseline at 75.0000%) with a 7.9% reduction in file size (3,584 B vs. 3,892 B) and a 20.8% reduction in active MACs (304 vs. 384).
- Student A (student_a_8_4_fp32): Achieved the smallest absolute storage footprint (2,976 Bytes, a 23.5% compression) and 160 MACs, maintaining a 71.6339% test accuracy.

Within the evaluated model family, the structurally smaller distilled students produced smaller serialized FlatBuffer artifacts than the unstructured pruned variants because distillation physically reduces tensor dimensions rather than masking elements to zero.

D. RQ4: Multi-Objective Deployment Pareto Frontier

Evaluating the 12 candidate models across the 3-objective deployment-resource space identifies exactly 6 Pareto-optimal configurations:

- student_b_16_4_fp32 (Top Accuracy): Accuracy = 75.14%, Size = 3,584 B, Active MACs = 304. Non-dominated because it achieves the global maximum test accuracy.
- pruned_mlp_14f_25pct (High-Accuracy Sparse): Accuracy = 75.05%, Size = 3,920 B, Active MACs = 288. Non-dominated because it requires fewer MACs (288) than student_b_fp32 while exceeding the accuracy of all other sparse models.
- pruned_mlp_14f_50pct (Balanced Sparse): Accuracy = 74.95%, Size = 3,920 B, Active MACs = 192. Non-dominated because it reduces active MACs (192) below pruned_25pct with higher accuracy than pruned_75pct.
- pruned_mlp_14f_75pct (Minimal Compute): Accuracy = 74.82%, Size = 3,920 B, Active MACs = 96. Non-dominated because it achieves the global minimum theoretical active MAC count.
- student_b_16_4_int8 (Quantized Distilled): Accuracy = 74.56%, Size = 3,576 B, Active MACs = 304. Non-dominated because it achieves a smaller serialized file size than student_b_fp32 (3,576 B vs. 3,584 B) with pure integer execution.
- student_a_8_4_fp32 (Minimal Storage): Accuracy = 71.63%, Size = 2,976 B, Active MACs = 160. Non-dominated because it achieves the global minimum serialized file size.

All remaining 6 models are strictly dominated. Importantly, removing host latency from the objective space does not alter the set of six Pareto-optimal configurations, confirming that the deployment-resource frontier is robust and mathematically consistent.

E. Secondary Host Execution Benchmark

As shown in Table I, single-sample host execution latencies on x86_64 range from 0.82 µs (student_b_fp32)

to 1.69 µs (pruned_25pct). These host measurements show relative operator dispatch overhead under the standard TFLite x86 runtime. However, since x86 architectures have deep superscalar pipelines, branch predictors, and multi-level caches which are absent on embedded microcontrollers, host timing should be regarded only as a secondary baseline, not a microcontroller execution metric.

F. Physical Microcontroller Deployment Validation

To ground the deployment characterization in physical hardware, the verified FULL_INT8 models were deployed to an ESP32-D0WD-V3 microcontroller (Xtensa LX6 dual-core @ 240 MHz, 4 MB Flash, 320 KB SRAM). Single-sample inference latency was captured via the hardware monotonic timer esp_timer_get_time() across 24,000 measured iterations (3 independent rounds ×2,000 inferences per model):

- student_a_8_4_int8 (176 params, 3,208 B): Mean = 64.55 µs, P95 = 69.00 µs, P99 = 76.00 µs, Max = 77 µs (Host/ESP32 Slowdown: 63.3×).
- student_b_16_4_int8 (328 params, 3,576 B): Mean = 72.96 µs, P95 = 83.00 µs, P99 = 83.00 µs, Max = 84 µs (Host/ESP32 Slowdown: 74.5×).
- mlp_12f_int8 (380 params, 3,712 B): Mean = 76.77 µs, P95 = 83.00 µs, P99 = 90.00 µs, Max = 90 µs (Host/ESP32 Slowdown: 76.8×).
- mlp_14f_int8 (412 params, 3,728 B): Mean = 89.90 µs, P95 = 95.00 µs, P99 = 101.00 µs, Max = 102 µs (Host/ESP32 Slowdown: 62.9×).

These physical measurements provide orthogonal system deployment latency evidence complementary to the primary accuracy–size–theoretical-active-MAC Pareto analysis. In our evaluated four-model set, knowledge distillation (student_a) achieves a physical inference latency reduction of 28.2% ($\frac{89.90-64.55}{89.90} \times 100\%$) versus mlp_14f on physical microcontrollers. Notably, physical microcontroller latency varies monotonically with parameter count (176 → 328 → 380 → 412 params), revealing arithmetic differences masked by host x86 superscalar pipelines while preserving the mathematical definition of the primary 3D Pareto frontier.

VII. Discussion

A. Why Multi-Objective Optimization is Essential in TinyML

In embedded ML design, choosing models based on a single metric like classification accuracy can lead to suboptimal solutions. For instance, though student_b_16_4_fp32 has the best classification accuracy (75.14%), pruned_mlp_14f_75pct requires only 31.6% of its arithmetic operations (96 vs. 304 MACs) with just 0.32% difference in classification accuracy. Since ultra-low-power microcontrollers benefit tremendously from reducing active arithmetic operations, especially during cyclic wake-up tasks, this 3× reduction in theoretical active arithmetic operations will bring significant energy savings.

B. Practical Deployment Decision Framework

Our empirical findings provide actionable design guidance for edge engineers navigating hardware constraints:

- When Flash storage is the primary constraint: Structural knowledge distillation (student_a_8_4_fp32) is the most space-efficient Pareto choice (2,976 B, a 23.5% compression).
- When active arithmetic demand is the primary concern: Magnitude pruning (pruned_mlp_14f_75pct) is the most compute-efficient Pareto choice (96 active MACs), subject to runtime sparse kernel support.
- When integer ALU execution is required: Full INT8 quantization (student_b_16_4_int8) provides pure integer execution graphs without floating-point emulation.
- When physical MCU execution latency is the primary concern: Physical on-device profiling confirms that compact student topologies achieve a 28.2% execution speedup on 32-bit Xtensa registers, requiring only 916 Bytes of allocator-committed tensor memory.

VIII. Threats to Validity

A. Internal Validity

All metrics were computed deterministically using fixed random seeds (seed = 42) and evaluated directly on disk-serialized FlatBuffer binaries. Data preprocessing and scaling parameters were fitted strictly on training data to prevent data leakage.

B. External Validity

The models were evaluated on automotive combustion engine telemetry. While physical sensor distributions vary across applications, the mathematical properties of affine quantization, dense FlatBuffer storage, and structural distillation hold across general tabular TinyML domains.

IX. Limitations

We explicitly state the following limitations:

1) Single Domain Scope: Evaluated on EngineFaultDB dataset; cross-domain generalization on audio and vibrations benchmarks is left for future work.
2) Model Architecture Family: Experiments are conducted on sub-4 KB Multi-Layer Perceptrons; convolutional and transformer family of architecture may have different compression ratios.
3) Serialization Format Dependence: Results are obtained for standard TensorFlow Lite FlatBuffer serialization; custom sparse storage formats (CSR/CSC) may achieve different results for sparse weights storage.
4) Empirical Latency vs. Formal WCET: While single-sample INT8 inference was physically profiled on ESP32 silicon (Obtained 64.55–89.90 µs Mean latency), these represent empirical distributions of latencies on devices and do not provide formal bounds on static Worst-Case Execution Time (WCET).

5) Absence of Physical Energy Profiling: We report theoretical active MACs and physical timer latencies; physical power dissipation in Joules/mW was not measured.
6) Absence of Structured Pruning Baseline: Pruning experiments evaluate unstructured magnitude pruning; structured channel-level pruning was not evaluated in this suite.
7) Sparse Acceleration Dependency: Standard TFLite runtime executes dense GEMM operations; realizing latency speedups from 75% pruning requires specialized sparse execution kernels.
8) Theoretical Compute vs. Hardware Cycles: Theoretical active MAC counts may not linearly map to hardware cycle counts due to instruction fetch and memory stall overheads.

X. Reproducibility and Artifact Availability

All experimental code, trained model FlatBuffers, dataset splits, physical ESP32 PlatformIO firmware, and verification scripts are available in the project repository. The model metrics can be verified by executing scripts/phase4_5_verification.py against results/tinyml_model_profile_verified.csv. Physical microcontroller deployment firmware is located under phase5/firmware/.

XI. Conclusion

This paper presented an empirical 3-objective deployment-resource Pareto characterization and low-level FlatBuffer artifact benchmark of model compression paradigms for ultra-low-resource TinyML sensor classification. By profiling 12 serialized TFLite models across quantization, feature reduction, unstructured pruning, and distillation, we established the accuracy–storage–compute Pareto frontier. Our results demonstrate that knowledge distillation achieves superior structural compression and accuracy (75.14% at 3,584 B), 75% pruning minimizes theoretical active MACs (96 MACs) without storage reduction, and full INT8 quantization achieves integer-only execution with zero float32 fallback. These verified findings provide actionable Pareto baselines for resource-constrained edge intelligence.

Acknowledgment

The authors thank the open-source TinyML and TensorFlow Lite Micro communities for developing accessible embedded machine learning runtimes.

References

[1] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. O'Reilly Media, Inc., 2019.
[2] P. P. Ray, "A review on tinyml: State-of-the-art and prospects," Journal of King Saud University-Computer and Information Sciences, vol. 34, no. 4, pp. 1595–1623, 2022.
[3] B. Mohammed, A. Al-Shaikhi, and J. Bell, "Tinyml for autonomous edge ai: A survey," IEEE Access, vol. 11, pp. 12 184–12 205, 2023.
[4] C. Banbury, V. J. Reddi, M. Lam, W. Fu, A. Fazel, J. Holleman, X. Huang, R. Hurtado, D. Kanter, A. Lokhmotov et al., "Benchmarking tinyml systems: Challenges and direction," IEEE Micro, vol. 41, no. 6, pp. 40–49, 2021.
[5] D. Dutta and P. K. Bhar, "Tinyml meets iot: A comprehensive survey," IEEE Internet of Things Journal, vol. 8, no. 23, pp. 16 603–16 624, 2021.
[6] R. Sanchez-Iborra and A. F. Skarmeta, "Tinyml-enabled frugal smart objects: Challenges and opportunities," IEEE Circuits and Systems Magazine, vol. 20, no. 3, pp. 4–18, 2020.
[7] R. David, P. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, D. Marculescu, M. Meng, P. Morales, J. Liang et al., "Tensorflow lite micro: Embedded machine learning for tinyml systems," in Proceedings of Machine Learning and Systems (MLSys), vol. 3, 2021, pp. 800–811.
[8] E. Liberis, Ł. Dudziak, and N. D. Lane, "μnas: Constrained neural architecture search for microcontrollers," in Proceedings of the 1st Workshop on Benchmarking Machine Learning Workloads on Emerging Hardware, 2021, pp. 1–7.
[9] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "Mcunet: Tiny deep learning on iot devices," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 11 711–11 722.
[10] J. Lin, W.-M. Chen, H. Cai, C. Gan, and S. Han, "Mcunetv2: Memory-efficient patch-based inference for tiny deep learning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 34, 2021, pp. 873–885.
[11] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," in Proceedings of the International Conference on Learning Representations (ICLR), 2016.
[12] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," Low-Power Computer Vision, pp. 291–326, 2022.
[13] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," in arXiv preprint arXiv:1704.04861, 2017.
[14] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 2704–2713.
[15] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," Proceedings of the IEEE, vol. 105, no. 12, pp. 2295–2329, 2017.
[16] D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, and J. Guttag, "What is the state of neural network pruning?" in Proceedings of Machine Learning and Systems (MLSys), vol. 2, 2020, pp. 129–146.
[17] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in Proceedings of the International Conference on Learning Representations (ICLR), 2019.
[18] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste, "Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks," The Journal of Machine Learning Research, vol. 22, no. 1, pp. 10 882–11 005, 2021.
[19] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv preprint arXiv:1503.02531, 2015.
[20] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," International Journal of Computer Vision, vol. 129, no. 6, pp. 1789–1819, 2021.
[21] A. Polino, R. Pascanu, and D. Alistarh, "Model compression via distillation and quantization," in Proceedings of the International Conference on Learning Representations (ICLR), 2018.
[22] C. Banbury, C. Zhou, I. Fedorov, R. Matas, U. Thakker, P. Whatmough, M. Mattina, and V. Janapa Reddi, "Micronets: Neural network architectures for deploying tinyml applications on commodity microcontrollers," in Proceedings of Machine Learning and Systems (MLSys), vol. 3, 2021, pp. 517–532.
[23] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once-for-all: Train one network and specialize it for efficient deployment," in Proceedings of the International Conference on Learning Representations (ICLR), 2020.